

FINAL PROJECT PRESENTATION

Sudoku Solver Agent using CSP & Backtracking

Artificial Intelligence CCE414 | May 2026

Ramy Said
231903795

David George
231903654

Ahmed Mohamed
232903959



Problem Formulation

Modeling Sudoku as a Constraint Satisfaction Problem (CSP) to achieve systematic and efficient solutions.

Defining the Logic

CSP Modeling

We treat Sudoku as a triple (X, D, C) :

- **Variables (X):** The 81 cells in the 9×9 grid.
- **Domains (D):** Numbers $\{1-9\}$ for empty cells.
- **Constraints (C):** Rules preventing repetition.

The Constraints

The solution must satisfy three strict rules:

- **Row:** No duplicate in any horizontal line.
- **Col:** No duplicate in any vertical line.
- **Block:** No duplicate in any 3×3 subgrid.

Search Problem Components



Initial State

The unsolved 9x9 board with some pre-filled numbers and empty spots represented as 0.



Actions

Placing a valid number (1-9) into an empty spot without violating Sudoku constraints.



Goal Test

All 81 spots are correctly filled, and all rules (Row, Col, Box) are strictly followed.

Intelligent Cell Selection

MRV Heuristic

The algorithm uses **Minimum Remaining Values (MRV)** to minimize search space via `get_mrv_cell`.

Scanning: Evaluates all 81 cells to calculate legal options (1-9) based on current constraints.

If no empty cells remain, the puzzle is solved.



Immediate Backtrack

If a cell has 0 options, the agent returns immediately, saving massive processing time.



Immediate Assignment

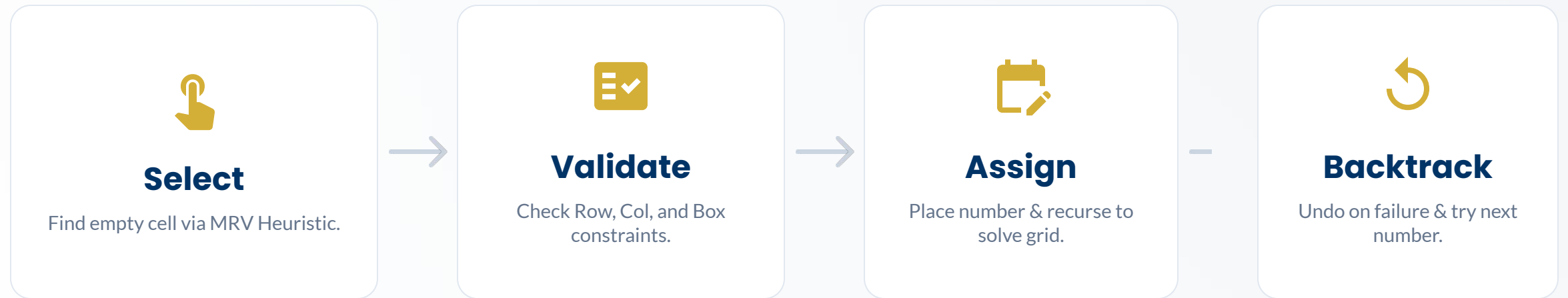
If a cell has exactly 1 option, it is selected instantly, skipping further board checks.



Default Action

If neither extreme is found, the agent selects the cell with the fewest available options.

Execution Workflow



This recursive Depth-First Search intelligently navigates the state space to find the valid configuration.

Performance Breakthrough

78x
Speed Increase

The "Evil Grid" Case Study

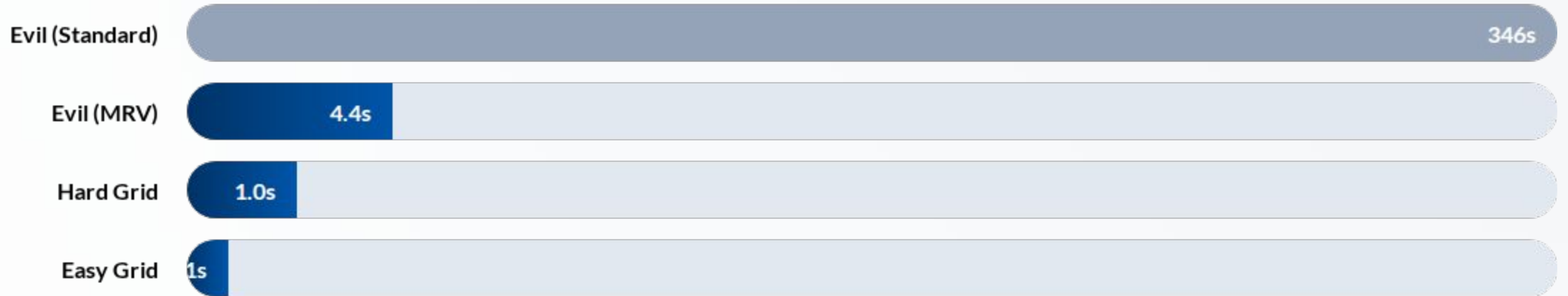
Standard backtracking gets trapped in millions of combinations for highly constrained grids. By implementing the **Optimized MRV Heuristic**, our agent identifies critical cells immediately.

- **Standard:** 346.13 seconds (~5.7 minutes)
- **Optimized:** 4.41 seconds

Experimental Results

Puzzle Complexity	Grid Type	Standard Time (s)	MRV Optimized (s)	Status
Low	Easy Grid	0.00062	0.00067	Solved
Medium	Medium Grid	0.00780	0.00830	Solved
High	Hard Grid	1.02100	1.01359	Solved
Extreme	Evil Grid	346.131	4.41608	Solved

Solving Time Comparison



The heuristic overhead is negligible for easy grids, but becomes critical as complexity scales exponentially.

Free Topic: Local Beam Search

Parallel Heuristic Search

Instead of one path, LBS keeps track of **k states** simultaneously.



Beam Width (k)

The number of best states kept in memory.



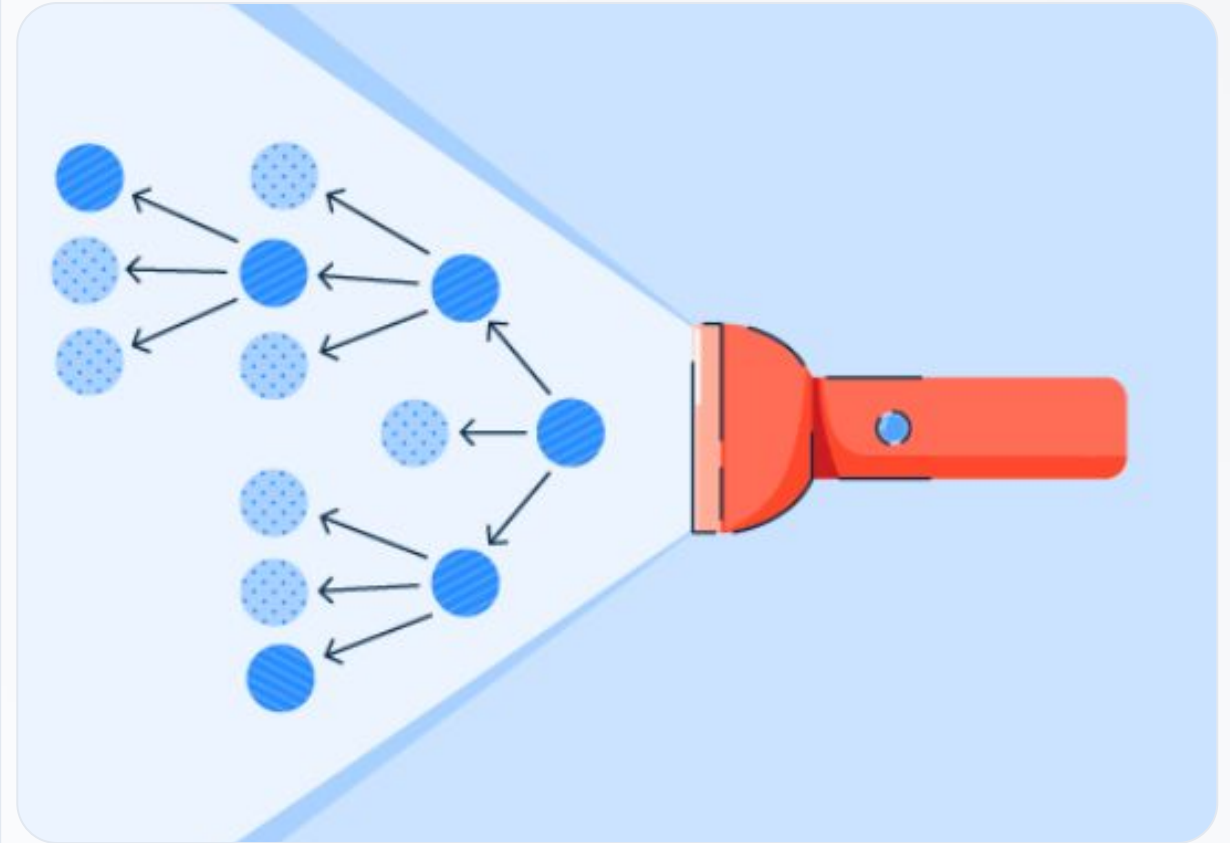
Collaboration

The k states "talk" by picking the absolute best successors from the entire pool.



Memory Efficient

Avoids saving the full search tree like BFS.



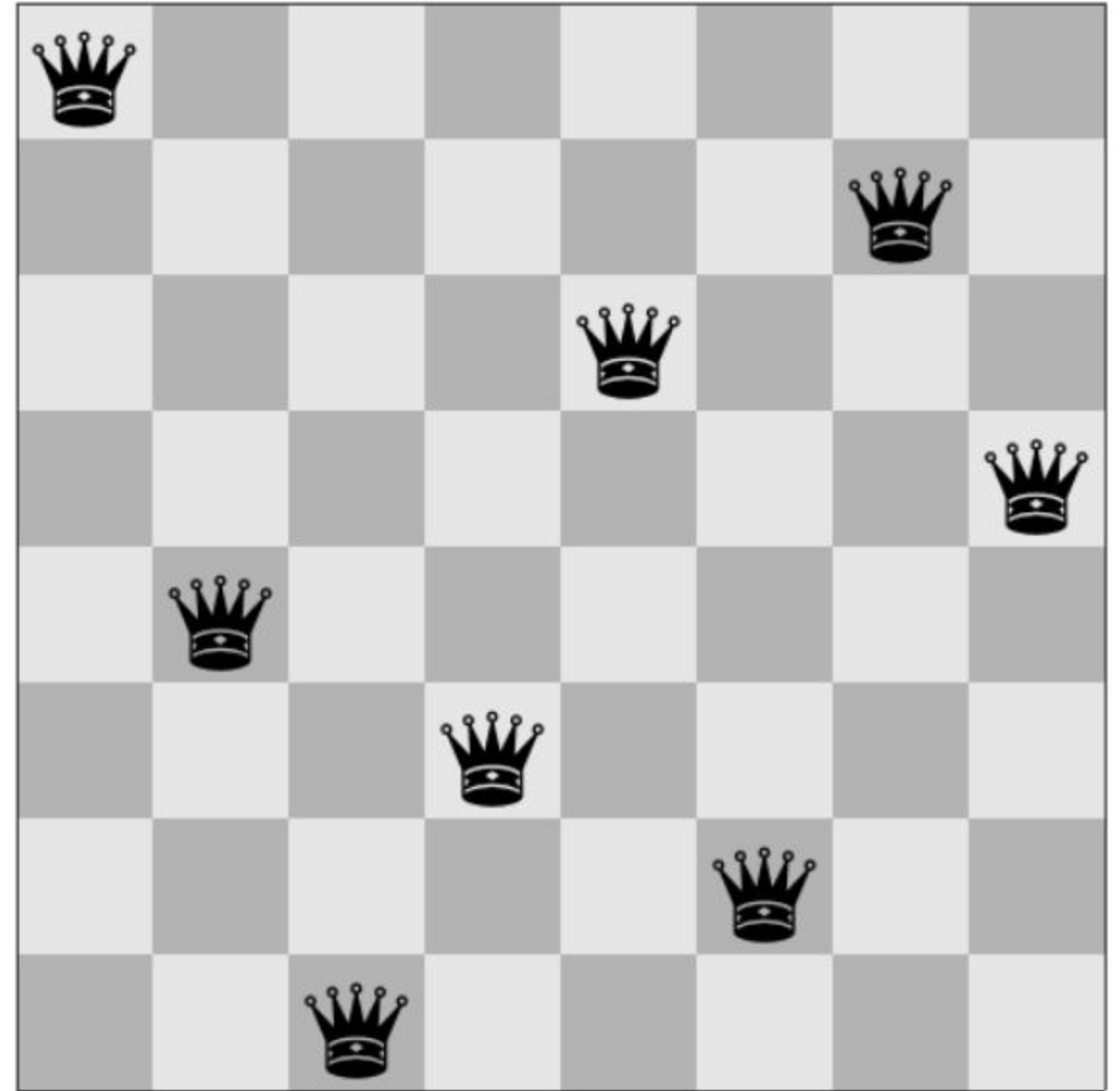
Case: 8-Queens Problem

Algorithm Execution

A classic chess puzzle where 8 queens must be placed on an 8x8 board with zero attacking conflicts.

LBS generates 224 neighbors from 4 initial beams ($k=4$), selects the top 4 based on conflict scores, and iterates until $h=0$.

Result: Rapid convergence compared to standard hill-climbing.



Thank You!

Questions?

Building this AI agent demonstrated that selecting the right heuristic is essential for navigating strict constraint state-spaces like Sudoku.

Artificial Intelligence | Engineering CCE414

5 / 10 / 2026